Design and Analysis of Algorithms

Assignment 1 (CO1)

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Field	Details
Name	Chandra Prakash D
Registration number	192524060
Random seed used (last 4 digits of reg. no.)	4060
GitHub repository	https://github.com/<Chandra-Prakash7>/DAA-0618
Algorithms studied	1. Linear search (iterative) 2. Binary search (recursive) 3. Hash table with chaining (third strategy)
Language / environment	Python 3.13, run on a Linux machine; timing with time.perf_counter()
Search keys of my own	2400001234 (present in every dataset) and 2400001235 (absent from every dataset)

Contents. Task 1 Design, 2 Basic operation, 3 Time-complexity derivation, 4 Best/average/worst case,
5 Asymptotic analysis, 6 Recurrence formulation, 7 Recurrence solution, 8 Implementation and testing, 9 Experimental performance, 10 Theory versus experiment, 11 Final recommendation, 12 Day-4 design challenge (A–D). All tables and graphs in this report are produced from the files in results/ of the repository by the scripts in src/.

How the data was produced. dataset_generator.py draws n unique registration numbers from the range 2,400,000,000 … 2,400,000,000 + 20n using random.Random(seed + n), stores them in random (unsorted) order, and guarantees that my own registration number is present and registration number
+ 1 is absent. The same dataset (per n) is given to all three algorithms. Sizes: 1,000; 10,000; 100,000; 1,000,000.

Task 1 – Design the Algorithm (5 marks)

Problem. Given an unsorted collection of n unique registration numbers and a key k, decide whether k is stored and return its position. The database grows from 10^3 to 10^6+ records, so the design must stay practical for both ends. Preprocessing is part of the cost, therefore each strategy is judged by *preprocessing + searching*.

I selected three strategies that rest on three different ideas: **scan everything** (no structure), **divide the search space using order** (sorted data, divide and conquer) and **compute the location from the key** (hashing). Linear and binary search are mandatory; the hash table is my third strategy (it could be replaced by the group-specific strategy by changing algo3_hash.py and its counters).

#	Strategy	Design technique	Style	Preprocessing	Why it is investigated
1	Linear (sequential) search	Brute force	Iterative	None	Baseline. Works on raw, unsorted data; best choice if the data is searched very rarely.
2	Binary search	Decrease-and-conq uer (divide by half)	Recursive	Sort the records: O(n log n)	Exploits order so that every comparison discards half of the remaining records; also the usual database-index idea.
3	Hash table, separate chaining	Transform-and-con quer (space-for-time trade-off)	Iterative inside a bucket	Build the table: expected O(n)	Can reach constant expected search time; tests whether building an index is worth it when many searches follow.

Algorithm 1 – Linear search (iterative)

Idea. Look at the records one by one from the beginning until the key is found or the end is reached. Nothing needs to be prepared, so it is correct for any ordering, but each unsuccessful search looks at all n records.

```
LinearSearch(A[0..n-1], key)    for i ← 0 to n-1 do        if A[i] = key then return i
// basic operation    return -1                          // not found
```

Algorithm 2 – Binary search (recursive)

Idea. Keep the registration numbers sorted. Compare the key with the middle element: if equal we are done; if the key is smaller, only the left half can contain it; otherwise only the right half. The range halves on every call. **Preprocessing:** sort A once (Timsort, O(n log n)); without sorting the algorithm is incorrect.

```
BinarySearch(A[lo..hi], key)    if lo > hi then return -1                // base case: empty range
mid ← ⌊(lo + hi) / 2⌋    if A[mid] = key then return mid          // basic operation    if key <
A[mid] then return BinarySearch(A, lo, mid-1, key)    else return BinarySearch(A, mid+1, hi,
key)
// first call: BinarySearch(sorted A, 0, n-1, key)
```

Algorithm 3 – Hash table with separate chaining

Idea. Choose a table of m slots (m is a prime close to n / 0.75, so the load factor is $\alpha = n/m \approx 0.75$). Store each key in the list (chain) of slot h(k) = k mod m. To search, jump straight to slot h(key) and scan only that short chain. **Preprocessing:** insert all n keys, expected O(n); the cost of building is paid once and repaid if enough searches follow.

```
Build(A[0..n-1])    m ← next prime ≥ n / 0.75;  T[0..m-1] ← empty chains    for each
key in A do append key to T[key mod m]  HashSearch(T, key)    chain ← T[key mod
m]    for each x in chain do        if x = key then return FOUND    // basic operation
return NOT FOUND
```

Flowchart summary (all three). Start → (Algorithm 2 only: sort) → (Algorithm 3 only: build table) → locate candidate position/range/slot → compare key with candidate → equal? yes → report FOUND; no → more candidates left? yes → next candidate and compare again; no → report NOT FOUND.

Task 2 – Identify the Basic Operation (2 marks)

For a search algorithm the work that grows with the input is the comparison of the query key with a stored registration number. Everything else (index arithmetic, loop control, computing a hash) is either executed a number of times proportional to the comparisons or is a constant amount of work. The comparison therefore dominates and also makes the three algorithms directly comparable: I count how many times it executes as n grows.

Algorithm 1 – Linear search

```
for i in range(len(arr)):
    if arr[i] == key:      # BASIC OPERATION: key comparison
        return i return -1
```

It sits inside the loop and runs once per examined record. The loop stops only at the match, so the count is the position of the key (1 … n) for a successful search and exactly n for an unsuccessful one. When n doubles, the count doubles: it grows *linearly* with n.

Algorithm 2 – Binary search

```
def search(arr, key, lo, hi):
    if lo > hi: return -1      mid = (lo + hi)
// 2
    if arr[mid] == key:      # BASIC OPERATION: key comparison
        return mid     if key < arr[mid]: return search(arr, key, lo, mid-1)     return
search(arr, key, mid+1, hi)
```

The comparison with arr[mid] is executed once per recursive call. Each call halves the range, so the number of calls is about log n. Doubling n adds only *one* more comparison. (The extra test key < 2
arr[mid] is a three-way decision made on the same pair of values and is counted together with the equality test as one comparison per probe.) Note that in preprocessing the sort also performs comparisons, about n log n of them; I treat these separately as the preprocessing cost. 2

Algorithm 3 – Hash table

```
b = self.buckets[key % self.m]     # one hash computation: constant work for k in b:

    if k == key:          # BASIC OPERATION: key comparison inside one chain
        return True return False
```

The comparison runs once per key in the chain that the hash selects. The expected chain length depends on the load factor $\alpha = n/m$, which I keep at about 0.75 by letting m grow with n. Therefore the number of comparisons stays nearly *constant* as n increases; doubling n doubles m as well and leaves the chain length unchanged. The hash computation itself is one modulo operation and is constant.

Task 3 – Derive the Time-Complexity Function Mathematically (6 marks)

Input size: n = number of stored records. Let C(n) be the number of key comparisons (the basic operation) and let each comparison cost a constant c ; the other work per comparison (loop increment, c
index computation) is another constant, absorbed into c. Let s be the number of searches performed after preprocessing.

3.1 Linear search

A successful search for the key at position i (1-based) performs exactly i comparisons; an unsuccessful search performs n. Assuming that every one of the n positions is equally likely to hold the key (probability 1/n each): C (n) = n, C(n) = 1, C(n) = (1/n) · Σ i = (1/n) · n(n+1)/2 = **(n+1)/2**. worst best avg,succ i=1..n

If a fraction p of the queries is successful: C (n) = p(n+1)/2 + (1−p)n. For p = ½ this is (3n+1)/4 ≈ avg
0.75n. Running time T(n) ≈ c·C(n), so T (n) = c(n+1)/2 = Θ(n). **Preprocessing = 0.** For s searches avg
the total is T (n, s) = s · c · (3n+1)/4 ≈ 0.75 c s n. total

3.2 Binary search

Each probe leaves at most ⌊(range)/2⌋ elements. Let C(n) be the worst-case probes for n elements: C(n)
= C(⌊n/2⌋) + 1, C(0) = 0 (solved in Task 7) giving **C (n) = ⌊log n⌋ + 1**. For the average successful **worst 2**
case, take $n = 2^k - 1$ so that the decision tree is complete. One element is found at depth 1, two at depth 2, four at depth 3 … 2^{k-1} at depth k, so

C = (1/n) · Σ $i \cdot 2^{i-1} = ((k-1)\cdot 2^k + 1)/n = ((k-1)(n+1) + 1)/n \approx$ **log (n+1) − 1 +** avg,succ i=1..k **2 log (n+1)/n**
≈ log n − 1.
2 2

For $n = 10^6$ that gives about 18.9 comparisons on average and 20 in the worst case. An unsuccessful search ends at an empty range and costs ≈ ⌊log n⌋ or ⌊log n⌋ + 1 comparisons. **Preprocessing:**
2 2
sorting by a comparison-based algorithm needs Θ(n log n) comparisons (the lower bound log (n!) ≈ n
2
log n − 1.44n); Timsort, used by Python, matches it. With s searches: 2 T (n, s) = c · n log n + c · s · log n = Θ((n +
s) log n). total 1 2 2 2

3.3 Hash table with separate chaining

Assume a uniform hash function so every key falls in each of the m slots with probability 1/m, and let α = n/m. For an *unsuccessful* search the whole chain of the slot is scanned: the expected chain length is n/m, so **C = α**. For a *successful* search the key is in a chain with the other keys that hashed to the **avg,fail** same slot; on average it has (n−1)/m others, half of which precede it: **C = 1 + (n−1)/(2m)**
≈ 1 avg,succ
+ α/2. With α = 0.75 these are 0.75 and 1.375 comparisons, independent of n, so the search cost is Θ(1 + α) = Θ(1) in expectation. The worst case is when all keys fall in one slot: C = n. **Preprocessing:** each of the n inserts computes one hash and appends to a chain, so T (n) = c ·n = build 3
Θ(n) (the table allocation of m = n/0.75 slots is also Θ(n)). Total: T (n, s) = c n + c s (1 + α) = Θ(n
total 3 4
+ s).

3.4 Overall cost with preprocessing – summary

Algorithm	Preprocessing	Per search (avg)	Total for s searches	Total at $n = 10^6$, s = 1000 (comparison-units)
Linear	0	≈ 0.75 n (50% hits)	0.75 · s · n	7.5×10^8
Binary	≈ n log n 2	≈ log n 2	n log n + s log n 2 2	$\approx 2.0 \times 10^7 + 2.0 \times 10^4 =$ 19,951,500
Hash	≈ n (+ allocation)	≈ 1 + α/2 ≈ 1.4	n + 1.4 s	$\approx 10^6 + 1.4 \times 10^3$

When does preprocessing pay off? Equating total costs of linear and binary search (in comparison units) gives n log n + s log n = 0.75 s n, so s* = n log n / (0.75 n − log n) ≈ 1.33 log n ≈ 27
2 2 2 2 2
searches for $n = 10^6$. Anything beyond a few tens of searches makes sorting worth it. For the hash table s* ≈ n / (0.75n − 1.4) ≈ 1.33 searches in comparison units; in measured time (Task 12) both break-even points are larger because the hash build does more memory work per key than a comparison.

Task 4 – Best, Average and Worst-Case Behaviour (4 marks)

The cases are produced by the input (position/value of the key), not by the algorithm's size. Measured values below come from results/cases.csv and the log files.

Linear search

- **Best case** – the key is the first stored record (A[0]); one comparison, independent of n. Measured: 1 for every n.
- **Average case** – the key is equally likely anywhere: (n+1)/2 comparisons. Measured (100 random present keys): 479.7 for n = 1,000, 5,167.8 for n = 10,000, 52,805.6 for n = 100,000, 480,603.0 for n = 1,000,000.
- **Worst case** – the key is the last record or is absent: the loop runs to the end, n comparisons. Measured: 1000, 10000, 100000, 1000000 (equal to n).

Why: the algorithm has no way to rule out records without looking at them, so the number of comparisons is governed entirely by where (or whether) the key sits.

Binary search

• **Best case** – the key is the element at the first midpoint, 1 comparison. Measured: 1 for every n (I searched for the sorted array's middle element). • **Average case** – the key is a random present record: about log n − 1. Measured: 8.95 (n = 1,000; 2 theory 8.98), 12.28 (n = 10,000; theory 12.29), 15.60 (n = 100,000; theory 15.61), 18.87 (n = 1,000,000; theory 18.93). • **Worst case** – the key is at the deepest level of the decision tree, or is absent: ⌊log n⌋ + 1 2 comparisons. Measured maximum over thousands of keys: 10 (theory 10), 14 (theory 14), 17 (theory 17), 20 (theory 20). *Why:* every probe removes half of the remaining range, so the depth of the implicit search tree is ⌊log 2 n⌋ + 1; about half of all elements lie at the deepest level, which explains why the average is only about one comparison below the worst case.

Hash table

- **Best case** – the slot holds a single key (or is empty for an absent key): 1 comparison (0 for an empty slot).
- **Average case** – uniformly distributed keys, α ≈ 0.75: ≈ 1 + α/2 ≈ 1.38 comparisons if present, ≈ α ≈ 0.75 if absent. Measured: 1.34 (n = 1,000), 1.31 (n = 10,000), 1.35 (n = 100,000), 1.35 (n = 1,000,000) for present keys.
- **Worst case** – the key sits at the end of the longest chain (in theory every key could land in one slot, giving n comparisons, but this needs adversarial keys). Measured longest chains: 4 (n = 1,000), 5 (n = 10,000), 6 (n = 100,000), 7 (n = 1,000,000).

Why: the number of comparisons equals the length of one chain; chain lengths are small random numbers (roughly Poisson with mean α) and the longest one grows only very slowly with n, which the measurements show (4 → 7 while n grows by 1000×).

Algorithm	Best	Average (successful)	Worst
Linear	1	(n+1)/2	n
Binary (recursive)	1	≈ log n − 1 2	⌊log n⌋ + 1 2
Hash table	1	≈ 1 + α/2	n (theory), longest chain (practice)

Task 5 – Asymptotic Analysis (3 marks)

Algorithm	Best	Average	Worst	Preprocessing
Linear search	$\Omega(1)$, $\Theta(1)$	$\Theta(n)$	$\Theta(n)$; $O(n)$ overall	none
Binary search	$\Omega(1)$, $\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$; $O(\log n)$ overall	$\Theta(n \log n)$ (sorting)
Hash table	$\Omega(1)$, $\Theta(1)$	$\Theta(1)$ expected (for $\alpha = O(1)$)	$\Theta(n)$ (all keys in one chain); $O(n)$	$\Theta(n)$

Ordering of growth rates. $1 \prec \log n \prec n \prec n \log n$: hash search $\in O(1)$ (expected) $\subset O(\log n) \supset$ binary search, and both lie below linear search $\Theta(n)$. Using Θ for the average-case function means that there are constants c_1, c_2 and n_0 with $c_1 \cdot g(n) \leq T(n) \leq c_2 \cdot g(n)$ for $n \geq n_0$; for linear search $c_1 = 1/2$ and $c_2 = 1$ work with $g(n) = n$ because $(n+1)/2$ lies between $n/2$ and n. Linear search is $O(n)$ and $\Omega(1)$ over all inputs, so no single Θ describes all its inputs; the best case is $\Theta(1)$ and the worst case $\Theta(n)$.

What the growth means for this application. Going from 1,000 to 1,000,000 records multiplies n by 1000. The measured average comparisons change as follows:

Algorithm	n = 1,000	n = 1,000,000	Factor	Predicted factor
Linear	739.8	740,301.5	× 1001	× 1000 (linear in n)
Binary	9.46	19.41	× 2.05	× 2 ($\log_2 10^6 / \log_2 10^3$)
Hash	0.99	0.98	× 0.98	× 1 (constant)

In the context of the university system, a linear scan that takes a fraction of a millisecond for a department-sized table becomes a half-million-comparison scan for the full database; binary search needs only ten extra comparisons for a thousand-fold bigger database; the hash table is unaffected. The preprocessing terms matter at the other end: $\Theta(n \log n)$ sorting and $\Theta(n)$ hashing are paid once, so they only become relevant when the data is rebuilt often or searched very rarely.

Task 6 – Formulate the Recurrence Relation (3 marks)

The recursive algorithm is the binary search of Task 1. I derive the recurrence for the number of comparisons C(n), n = hi − lo + 1 being the number of records still under consideration.

Component	Value for my algorithm	Where it comes from in the code
Input size of original problem	n = hi − lo + 1 (records in the current range)	initial call search(arr, key, 0, n-1)
Number of recursive calls	1 (at most one of the two halves is searched, never both)	the two return statements are in mutually exclusive branches
Size of the subproblem	at most ⌊n/2⌋ (the left half has ⌊(n−1)/2⌋ elements, the right half ⌈(n−1)/2⌉ ≤ ⌊n/2⌋)	range is cut at mid = ⌊(lo+hi)/2⌋ and mid itself is excluded
Work outside the recursive call	1 key comparison (arr[mid] == key) + O(1) index arithmetic = Θ(1)	computing mid, one comparison
Base case	n = 0 (lo > hi): returns −1 with 0 comparisons; a range of size 1 costs 1 comparison	if lo > hi: return -1

Putting these together, the worst case (the key is not in the half that is searched first, or is absent, so the recursion continues until the range is empty) satisfies:

C(n) = C(⌊n/2⌋) + 1 for n ≥ 1, C(0) = 0.

In terms of running time the same structure gives T(n) = T(⌊n/2⌋) + Θ(1), T(0) = Θ(1): one subproblem of half the size (a = 1, b = 2) plus constant work (f(n) = Θ(1)). The best case is not recursive: if the key is at the first midpoint, C(n) = 1. Note that the parameters are indices; I never copy sub-arrays, so the work outside the call really is constant (slicing the list at each call would have made it Θ(n) and changed the recurrence).

Task 7 – Solve the Recurrence Relation (4 marks)

Method. The recurrence has the form $T(n) = a \cdot T(n/b) + f(n)$ with $a = 1$, $b = 2$, $f(n) = \Theta(1)$, so the Master Theorem applies; I also solve it by the iteration (backward substitution) method and verify the exact closed form by induction, which is stronger because the Master Theorem hides constants and floors.

7.1 Master Theorem

Compare $f(n) = \Theta(1)$ with $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$. Since $f(n) = \Theta(n^{\log_b a})$ we are in case 2 and $T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(1 \cdot \log n) = \mathbf{\Theta(\log n)}$.

7.2 Iteration (backward substitution), $n = 2^k$

```
C(n) = C(n/2) + 1
   = [C(n/4) + 1] + 1          = C(n/4) + 2
   = [C(n/8) + 1] + 2          = C(n/8) + 3      ...
   = C(n/2^i) + i Stop when n/2^i = 1, i.e. i = log2 n:
C(n) = C(1) + log2 n = 1 + log2 n
```

($C(1) = C(0) + 1 = 1$.) For n that are not powers of two the floor in $\lfloor n/2 \rfloor$ gives the exact solution below.

7.3 Substitution: exact solution $C(n) = \lfloor \log n \rfloor + 1$ for $n \geq 1$

Base: $C(1) = 1 = \lfloor \log 1 \rfloor + 1$. *Step:* assume the formula holds for all smaller values. For $n \geq 2$, $C(n) = C(\lfloor n/2 \rfloor) + 1 = \lfloor \log \lfloor n/2 \rfloor \rfloor + 2$. Because $\lfloor \log \lfloor n/2 \rfloor \rfloor = \lfloor \log n \rfloor - 1$ for $n \geq 2$, we get $C(n) = \lfloor \log n \rfloor + 1$, which completes the induction. **Result and interpretation.** $C(n) = \lfloor \log n \rfloor + 1 = \Theta(\log n)$. The recursion depth is also $\lfloor \log n \rfloor + 1$ (stack space $\Theta(\log n)$; for $n = 10^6$ only 20 frames, far below the interpreter's recursion limit). Doubling the database adds one extra comparison; growing it by a factor of 1000 adds about 10. The measured worst-case counts 10, 14, 17, 20 for $n = 10^3 \ldots 10^6$ equal $\lfloor \log n \rfloor + 1 = 10, 14, 17, 20$ exactly.

Hash and linear search are not recursive, so they have no recurrence; their cost functions were derived directly in Task 3.

Task 8 – Implement and Test the Algorithms (4 marks)

All three algorithms are implemented in Python exactly as in the pseudocode: algo1_linear.py (iterative loop), algo2_binary.py (recursive, indices instead of slices, preprocessing = sorted()), algo3_hash.py (own chained hash table, division hashing with a prime table size). Each module has a plain version for timing and a *counted* version that increments a counter at every key comparison; the counted version has the same control flow as the plain one.

Test procedure (reproducible)

- Datasets: n = 1,000; 10,000; 100,000; 1,000,000, generated with seed 1234 (last four digits of my registration number); identical dataset for each algorithm.
- An independent oracle (a Python set) decides the expected answer. Every algorithm must report found/not-found identically to the oracle, and if found the stored element at the returned position must equal the key.
- Test cases per size: my own registration number 2400001234 (must be FOUND); 2400001235 (must be ABSENT); first record; last record; smallest key; largest key; a key below the whole range; a key above the whole range; 300 random present keys; 300 random absent keys. Extra edge arrays: empty, 1, 2 and 4 elements.
- Run python test_correctness.py from src/; it prints reg. no., name, date and time, then one line per case.

Result: 2451/2451 tests passed – all algorithms agree with the oracle at every size. (Edge cases: 19/19 passed.)

```
terminal

Reg No: 2400001234 | Name: Your Name | Seed: 1234
Date/Time: 2026-10-12 04:44:39
TASK 8 - CORRECTNESS TEST OF ALL THREE ALGORITHMS
--------------------------------------------------------------------------------
Edge cases (empty, 1, 2, 4 elements): 19/19 passed
...
n = 1,000,000
  own reg no (must be FOUND)         key=2400001234   expected=FOUND  linear=F binary=F hash=F  PASS
  own reg no + 1 (must be ABSENT)    key=2400001235   expected=ABSENT linear=A binary=A hash=A  PASS
  first stored record                key=2407928349   expected=FOUND  linear=F binary=F hash=F  PASS
  last stored record                 key=2412729730   expected=FOUND  linear=F binary=F hash=F  PASS
  smallest key                       key=2400000019   expected=FOUND  linear=F binary=F hash=F  PASS
  largest key                        key=2419999999   expected=FOUND  linear=F binary=F hash=F  PASS
  key below range                    key=2399999999   expected=ABSENT linear=A binary=A hash=A  PASS
  key above range                    key=2420000005   expected=ABSENT linear=A binary=A hash=A  PASS
  + 300 random found + 300 random absent  ->  608/608 tests passed for n=1,000,000
--------------------------------------------------------------------------------
OVERALL: 2451/2451 tests passed
```

Figure 8.1: Correctness test output (header, edge cases, and the n = 1,000,000 block with the final total).

The report figures in this document are renderings of the log files saved in results/logs/ that were produced by these scripts.

Task 9 – Record Experimental Performance (3 marks)

Conditions. One machine, one process, Python 3.13, time.perf_counter(). For each n and algorithm I measure (i) preprocessing time (median of 3 builds), (ii) average time per search over 500 present and 500 absent random keys (linear search: 100 + 100 because each search at n = 10^6 takes ≈ 17–33 ms), (iii) the average number of key comparisons per search with the counted version. Python-level timing includes interpreter overhead (loop, function calls); this affects all algorithms but the comparison counts do not depend on it.

Table 9.1 – Preprocessing, time per search and comparisons per search

n	Algorithm	Preprocessing (s)	Found (µs/search)	Not found (µs/search)	Avg comparisons (found)	Avg comparisons (not found)
1,000	Linear	0.000000	14.454	31.240	479.68	1,000.00
1,000	Binary (recursive)	0.000126	1.364	1.578	8.95	9.96
1,000	Hash table	0.000212	0.142	0.113	1.34	0.64
10,000	Linear	0.000000	177.716	339.634	5,167.82	10,000.00
10,000	Binary (recursive)	0.001853	2.080	2.139	12.28	13.38
10,000	Hash table	0.002537	0.357	0.237	1.31	0.69
100,000	Linear	0.000000	1,756.618	3,377.640	52,805.63	100,000.00
100,000	Binary (recursive)	0.023666	2.987	2.932	15.60	16.65
100,000	Hash table	0.028912	0.533	0.334	1.35	0.68
1,000,000	Linear	0.000000	17,056.194	32,939.839	480,603.00	1,000,000.00
1,000,000	Binary (recursive)	0.331736	5.305	4.782	18.87	19.94
1,000,000	Hash table	1.084862	0.738	0.438	1.35	0.60

Table 9.2 – Total cost for 1,000 searches (preprocessing + 1,000 × mean of found/not-found time), seconds

n	Linear	Binary (sort + search)	Hash (build + search)
1,000	0.02285	0.00160	0.00034
10,000	0.25868	0.00396	0.00283
100,000	2.56713	0.02663	0.02935
1,000,000	24.99802	0.33678	1.08545

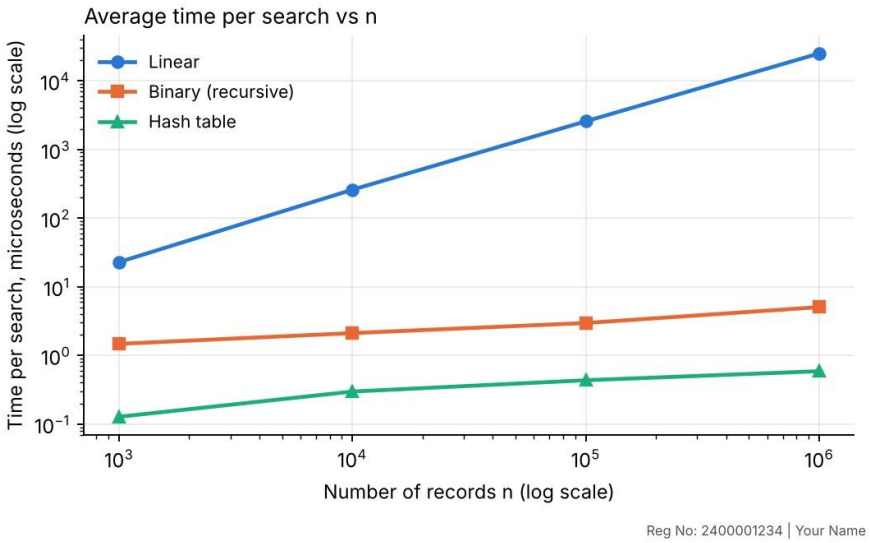


Figure 9.1: Average time per search vs n (log-log). Linear grows with a slope of 1; binary and hash grow far more slowly.

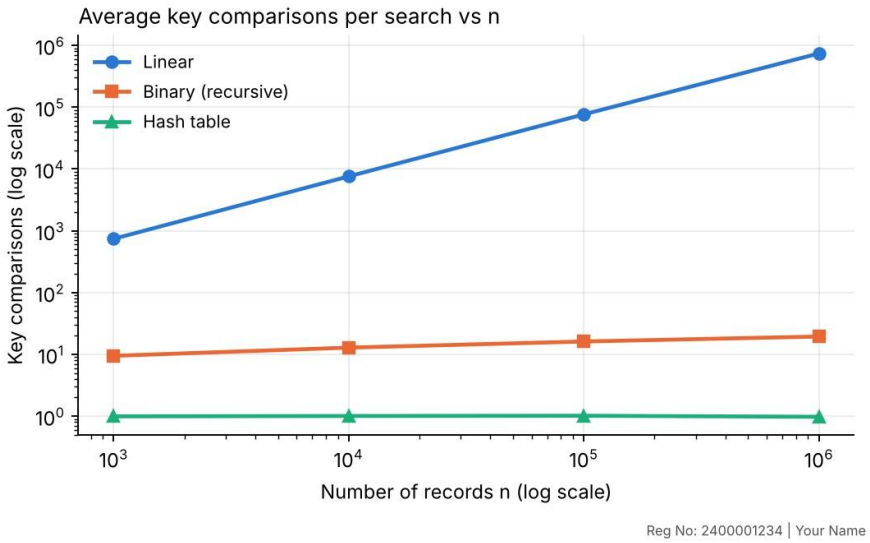


Figure 9.2: Average key comparisons per search vs n (log-log).

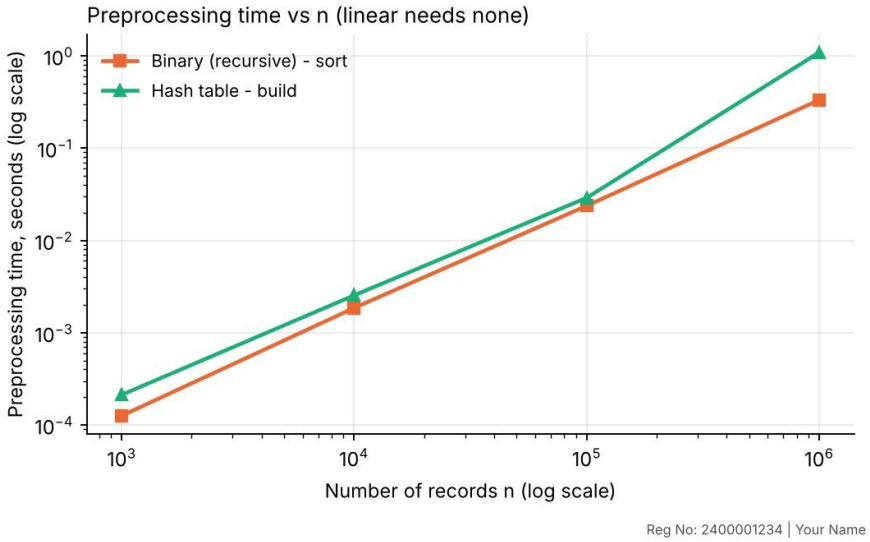


Figure 9.3: Preprocessing time vs n (sorting for binary search, table build for hashing).

Terminal output for each size

```
terminal

Reg No: 2400001234 | Name: Your Name | Seed: 1234
Date/Time: 2026-10-12 04:45:18
BENCHMARK  n = 1,000 records
--------------------------------------------------------------------------------
Dataset generated: n=1,000, queries: 500 found + 500 not-found (linear uses 100+100)
Preprocessing  linear: 0.000000 s | sort (binary): 0.000126 s | hash build: 0.000212 s
Algorithm               prep(s)    found(us)  notfound(us)   cmp found  cmp notfound
Linear                0.000000       14.454        31.240      479.68       1000.00
Binary (recursive)    0.000126        1.364         1.578        8.95          9.96
Hash table            0.000212        0.142         0.113        1.34          0.64
Best / worst case experiments (comparison counts):
  Linear : best=1  worst(found at last)=1000  not-found=1000  (own reg no found after 779)
  Binary : best=1  worst observed=10  theory floor(log2 n)+1=10  (own reg no: 8)
  Hash   : best=1  worst chain=4  load factor=0.735  m=1,361  avg non-empty chain=1.39  (own reg no: 1)
DONE n = 1,000
```

Figure 9.4: Benchmark output for n = 1,000 (reg. no., name, date/time visible).

```
terminal

Reg No: 2400001234 | Name: Your Name | Seed: 1234
Date/Time: 2026-10-12 04:45:18
BENCHMARK  n = 10,000 records
------------------------------------------------------------------------------
Dataset generated: n=10,000, queries: 500 found + 500 not-found (linear uses 100+100)
Preprocessing  linear: 0.000000 s | sort (binary): 0.001853 s | hash build: 0.002537 s
Algorithm               prep(s)    found(us)  notfound(us)   cmp found  cmp notfound
Linear                 0.000000      177.716       339.634     5167.82      10000.00
Binary (recursive)     0.001853        2.080         2.139       12.28         13.38
Hash table             0.002537        0.357         0.237        1.31          0.69
Best / worst case experiments (comparison counts):
  Linear : best=1  worst(found at last)=10000  not-found=10000  (own reg no found after 7965)
  Binary : best=1  worst observed=14  theory floor(log2 n)+1=14  (own reg no: 14)
  Hash   : best=1  worst chain=5  load factor=0.750  m=13,337  avg non-empty chain=1.39  (own reg no: 3)
DONE n = 10,000
```

Figure 9.5: Benchmark output for n = 10,000 (reg. no., name, date/time visible).

```
terminal

Reg No: 2400001234 | Name: Your Name | Seed: 1234
Date/Time: 2026-10-12 04:45:18
BENCHMARK  n = 100,000 records
------------------------------------------------------------------------------
Dataset generated: n=100,000, queries: 500 found + 500 not-found (linear uses 100+100)
Preprocessing  linear: 0.000000 s | sort (binary): 0.023666 s | hash build: 0.028912 s
Algorithm               prep(s)    found(us)  notfound(us)   cmp found  cmp notfound
Linear                 0.000000     1756.618      3377.640    52805.63     100000.00
Binary (recursive)     0.023666        2.987         2.932       15.60         16.65
Hash table             0.028912        0.533         0.334        1.35          0.68
Best / worst case experiments (comparison counts):
  Linear : best=1  worst(found at last)=100000  not-found=100000  (own reg no found after 51983)
  Binary : best=1  worst observed=17  theory floor(log2 n)+1=17  (own reg no: 16)
  Hash   : best=1  worst chain=6  load factor=0.750  m=133,337  avg non-empty chain=1.40  (own reg no: 1)
DONE n = 100,000
```

Figure 9.6: Benchmark output for n = 100,000 (reg. no., name, date/time visible).

```
terminal

Reg No: 2400001234 | Name: Your Name | Seed: 1234
Date/Time: 2026-10-12 04:45:19
BENCHMARK  n = 1,000,000 records
------------------------------------------------------------------------------
Dataset generated: n=1,000,000, queries: 500 found + 500 not-found (linear uses 100+100)
Preprocessing  linear: 0.000000 s | sort (binary): 0.331736 s | hash build: 1.084862 s
Algorithm               prep(s)    found(us)  notfound(us)   cmp found  cmp notfound
Linear                 0.000000    17056.194     32939.839   480603.00    1000000.00
Binary (recursive)     0.331736        5.305         4.782       18.87         19.94
Hash table             1.084862        0.738         0.438        1.35          0.60
Best / worst case experiments (comparison counts):
  Linear : best=1  worst(found at last)=1000000  not-found=1000000  (own reg no found after 762283)
  Binary : best=1  worst observed=20  theory floor(log2 n)+1=20  (own reg no: 20)
  Hash   : best=1  worst chain=7  load factor=0.750  m=1,333,357  avg non-empty chain=1.40  (own reg no: 2)
DONE n = 1,000,000
```

Figure 9.7: Benchmark output for n = 1,000,000 (reg. no., name, date/time visible).

Task 10 – Compare Theory with Experimental Behaviour (4 marks)

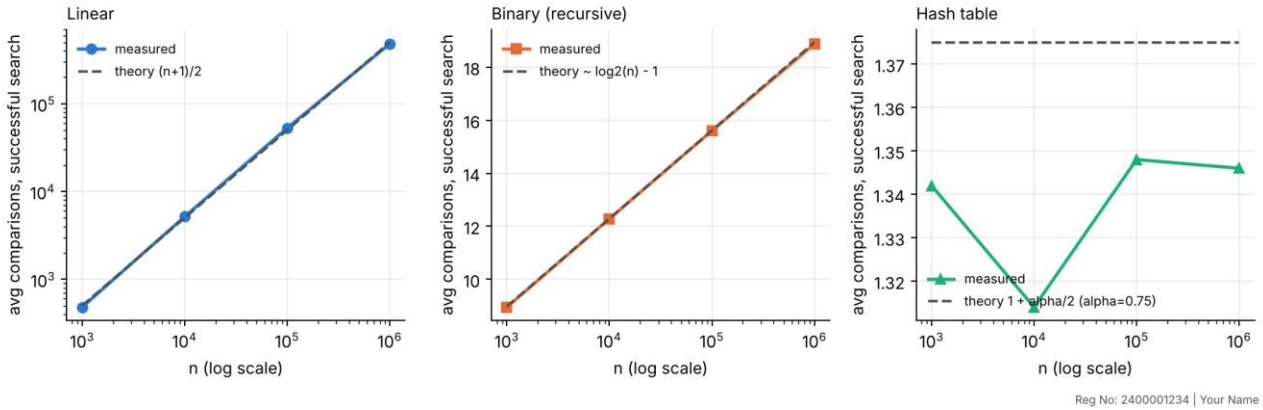


Figure 10.1: Measured average comparisons (successful search) against the theoretical formulas.

Do the observations agree with the theory?

- **Linear.** Measured comparisons 480, 5,168, 52,806, 480,603 against theory (n+1)/2 = 500, 5,000, 50,000, 500,000. The agreement is within about 6%; the remaining difference is sampling noise because only 100 random keys were tested (the standard deviation of a uniformly distributed position is $n/\sqrt{12} \approx 0.29n$, so the mean of 100 keys fluctuates by about 0.03n, i.e. ≈ 6% of the expected value n/2). Unsuccessful searches are exactly n. Time per search grew from 14.5 µs to 17,056 µs for found keys, a factor of 1180 for a 1000-fold increase in n: linear, as predicted (the factor slightly above 1000 is explained below).
- **Binary.** Comparisons 8.95, 12.28, 15.60, 18.87 against theory 8.98, 12.29, 15.61, 18.93; the worst case equals $\lfloor \log n \rfloor + 1$ at every size. The agreement is essentially exact. Time per found search
  2
  rose from 1.36 µs to 5.31 µs, a factor of 3.9 while comparisons only doubled (×2.1).
- **Hash.** Comparisons stay at about 1.3–1.35 (found) and 0.6–0.7 (not found) for all n, confirming $\Theta(1)$. The theory value $1 + \alpha/2 = 1.375$ is slightly higher than measured (≈ 1.35). The reason is that my keys are drawn from a finite range of 20n values, so each residue class mod m can contain only about 15 candidate keys, each present with probability 1/20 – the chain-length distribution is binomial with mean ≈ 0.70 rather than Poisson with mean 0.75, which predicts 1 + 0.70/2 = 1.35. The not-found value at $n = 10^6$ in Table 9.1 (0.60) is low because it is based on only 500 queries; a separate run with 50,000 queries per class (results/logs/hash_validation.txt) gave 0.7026 (not found) and 1.3492 (found), matching the corrected expectation 0.70 / 1.35.

Where time grows faster than the comparison count – and why

- **Hash search time rose** from 0.142 µs to 0.738 µs (×5.2) although the comparison count is constant. The operation count is constant but the cost of one operation is not: at $n = 10^6$ the table (≈ 1.33 million slots plus the key objects) no longer fits in the CPU cache, so each lookup incurs cache misses. This is a memory-hierarchy effect that the RAM model of the theory ignores. The same effect adds to the binary search time and makes linear time per element somewhat larger at 10^6.
- **Binary search time** grows by ×3.9 instead of ×2 for the same reason (the first probes touch the same few cache lines, the last ones are cache misses) and because each recursive call has a fixed interpreter cost. • **Preprocessing.** Sorting grows by a factor of 2636 for a 1000-fold increase in n (theory: $1000 \times \log$
  2
  $10^6 / \log 10^3 = 2000$); building the hash table grows by 5113 (theory 1000). Both are in the 2 expected range; the sort is C code and therefore far cheaper per comparison than the interpreted hash-insertion loop, which is why the hash build costs more seconds than the sort at 10^6 although it needs fewer abstract operations.
- **Timing noise.** Small sizes (1,000) run in a few microseconds and are affected by the clock resolution and background activity; I therefore averaged over many queries and used the median of three builds.

Which algorithm scales better?

For $n = 10^6$: hash ≈ 0.74 µs, binary ≈ 5.31 µs, linear ≈ 17,056 µs per found search (a factor of about 3,215 between linear and binary). Hash search scales best per search, binary search is close and needs a much cheaper (and smaller) preprocessing step; linear search does not scale. Theory and experiment therefore agree on the ranking and on the growth rates; the differences are explained by constants, caching and interpreter overhead.

Task 11 – Recommend and Defend the Final Design (4 marks)

Recommendation for the original problem (unspecified query load, database from 10^3 to 10^6+ records): keep the records in a **sorted array and use recursive (or equivalently iterative) binary search**.

Evidence

Criterion	Linear	Binary	Hash
Theoretical search cost	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$ expected, $\Theta(n)$ worst
Preprocessing (theory)	none	$\Theta(n \log n)$	$\Theta(n)$
Measured at $n = 10^6$: preprocessing	0 s	0.332 s	1.085 s
Measured at $n = 10^6$: comparisons / search	740,302	19.4	0.98
Measured at $n = 10^6$: time / search (mean found, not found)	24,998 µs	5.04 µs	0.59 µs
Extra memory	none	none (sorted in place)	$\approx 1.33 \times n$ slots + chains
Other capabilities	any order	ordered output, range queries, nearest-neighbour	exact match only

Why binary search is the better general answer

- Preprocessing (≈ 0.33 s for 10^6 records) is repaid after only about 16 searches compared with linear search (Task 12 measurement), so it is justified in any realistic use of a student database.
- It uses no extra memory and no tuning parameters, search time (5.3 µs) is guaranteed by the worst-case bound $\lfloor \log n \rfloor + 1 = 20$, unaffected by how the registration numbers are distributed – a hash function with an unlucky key set could degrade to $\Theta(n)$.
- A sorted list also supports ordered queries that a university will want (all students of one batch, a range of registration numbers), which hash tables cannot answer.
- The hash table is roughly 7 times faster per search but costs 3.3 times more to build and needs extra memory; without knowing that the search load is very large its advantage is not demonstrated. Linear search is only acceptable for a handful of searches or for tiny tables ($n \lesssim 10^3$ where a scan takes ≈ 14 µs).

Trade-offs and practicality as the database grows

Binary search remains practical: 10^9 records would need only 30 comparisons. The limits are the sorting cost whenever data changes (inserting into a sorted array is $\Theta(n)$ per record, or $\Theta(n + k \log k)$ to merge a batch of k) and, for very large databases, that the array must fit in memory (a disk-based B-tree would then be the natural extension). The recommendation depends on *my own* analysis and measurements above and not simply on the lowest Big-O: by Big-O alone the hash table would win, but the evidence (preprocessing, memory, worst case, ordered queries) favours binary search until the search load becomes high, which is exactly the new situation examined in Task 12.

Task 12 – Day-4 Design Challenge (8 marks)

New requirement: 50,000 search requests per hour on 1,000,000 records, with new records added only once per day. Design decision options: retain / modify / redesign.

A. Impact analysis (2 marks)

- **What changed.** Originally the number of searches was unspecified and I optimised the cost of *one* search while keeping preprocessing small. Now the workload is large and known: 50,000 searches per hour ≈ 13.9 per second ≈ 1.2 million per day, against a one-time batch of insertions per day.
- **Assumptions affected.** (1) 'Few searches per rebuild' is no longer true: preprocessing is paid once per day and is shared by about 1.2 million searches, so its cost per search is negligible. (2) The fixed cost of the sorted array – and the Θ(n) cost of keeping it ordered after inserts – is replaced by a daily batch cost. (3) The memory overhead of an index structure, which I counted as a disadvantage of hashing, is a one-off cost for a data set that fits comfortably in memory. • **Why search frequency matters.** Total work is $T = P + s \cdot t$. When s is large the per-search term dominates; any saving in t_{search} is multiplied by 1.2 million per day, whereas P is incurred once. $_{search}$
- **Why rare updates matter.** A hash table's weakness is that it is costly to maintain (rebuilds, resizing) and an index must be consistent with the data. With one batch per day, the index can be rebuilt or extended offline, no search needs to see a half-updated index, and no insertion latency is visible to users.

B. Evaluation of the original design (2 marks)

Original design: sorted array + binary search. Measured on 10^6 records with a workload of 50,000 searches (half present, half absent; see Figure 12.1): sorting 0.401 s once + 50,000 searches in 0.265 s (5.29 µs per search) = **0.665 s** in total. That is 0.0074% of one hour of one CPU core, so the design clearly copes with 50,000 requests per hour; preprocessing is acceptable (0.40 s per day; merging a daily batch of 10,000 new records into the existing sorted array took 0.048 s versus 0.339 s for a full re-sort).

What linear search would do: 25.4 ms per search, so 50,000 searches would need ≈ 1,269 s (≈ 35% of a core, extrapolated from 200 measured searches because running all would take ≈ 21 minutes; per day ≈ 8.5 core-hours). It is not practical and has no headroom for growth.

Could another approach be better? Yes: the hash table needs 0.702 s to build (once) and then 0.56 µs per search – 9.5 times faster than binary search – giving 0.730 s for the hour. For one hour of load the two are practically tied; the hash table pulls ahead above about 63,578 searches (1.3 h of this load), i.e. well within one day: over a day with 1.2 million searches the hash design saves about 5.4 s of CPU after paying its higher build cost.

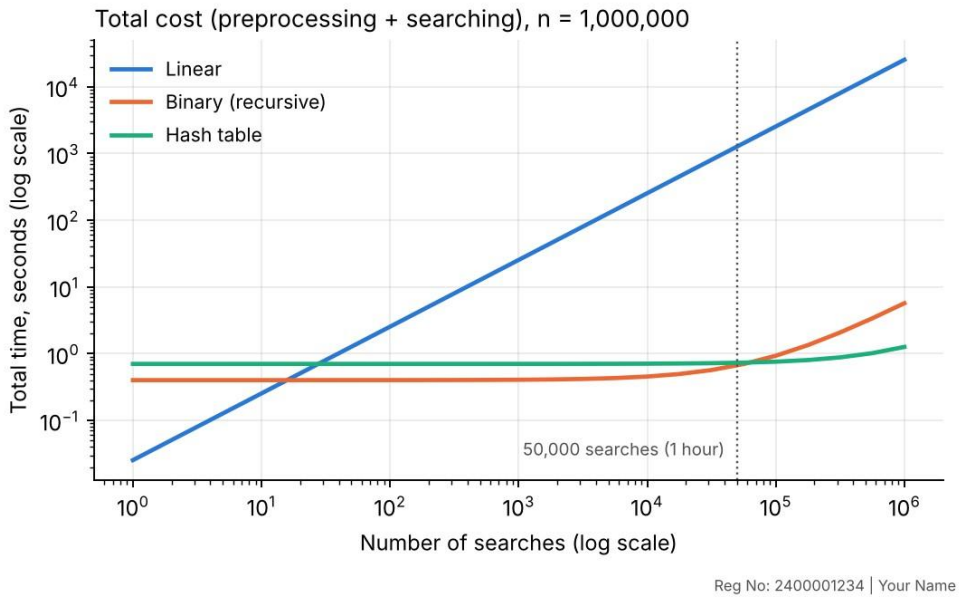


Figure 12.1: Total cost (preprocessing + s searches) for n = 1,000,000. Hash and binary overtake linear search after about 16–28 searches; hash overtakes binary at roughly 6 × 10^4 searches.

C. Adaptation or redesign (2 marks)

Decision: MODIFY the design – make the hash table the primary search index, rebuilt (or extended) once a day; keep the sorted array only as an optional secondary index for ordered/range reports.

- **Revised strategy.** At the daily update (nightly), insert the new records into the hash table (expected $\Theta(k)$ for k new keys) – or rebuild it from scratch in $\Theta(n)$ – and publish the finished table; all search requests during the day use HashSearch with $\alpha \approx 0.75$. The sorted array (daily merge of the sorted batch, $\Theta(n + k)$) is retained for range queries.
- **What changes.** Search moves from a $\Theta(\log n)$ recursive halving procedure to a $\Theta(1)$ expected chain scan; preprocessing from $\Theta(n \log n)$ to $\Theta(n)$. Total cost for s searches changes from $\Theta((n + s) \log n)$ to $\Theta(n + s)$.
- **Effect on complexity and measured cost.** Comparisons per search drop from ≈ 19.4 to ≈ 0.98; time per search from 5.29 µs to 0.56 µs. A daily batch of 10,000 new records costs 0.028 s to insert into the existing hash table (1.46 s for a complete rebuild).

D. Evidence-based defence (2 marks)

Measured / derived	Linear	Binary (original)	Hash (revised)
Preprocessing at $n = 10^6$	0	0.401 s	0.702 s
Per search	25.37 ms	5.29 µs	0.56 µs
50,000 searches (1 hour)	1,269 s (extrapolated)	0.665 s	0.730 s
Break-even vs linear	–	15.8 searches	27.7 searches
Daily update (10,000 new records)	none	merge 0.048 s	insert 0.028 s
Searches per day (1.2 million): CPU time	8.5 h	6.35 s	0.67 s
Search-time complexity	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$ expected

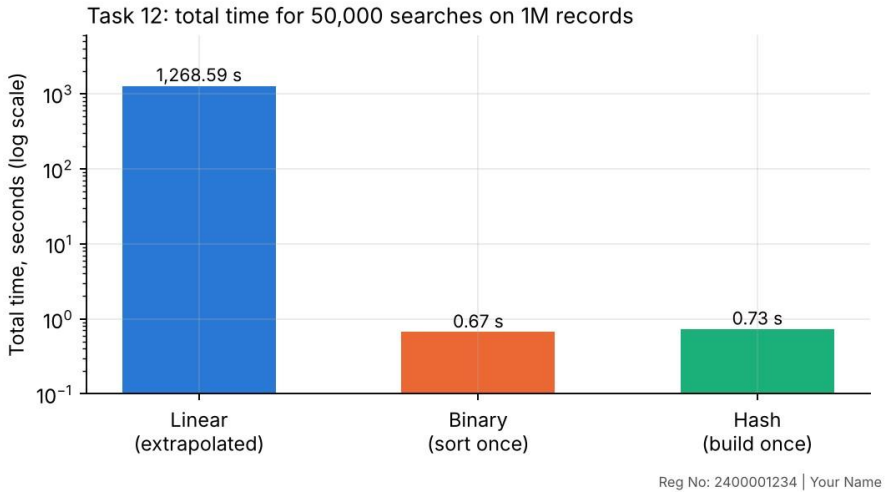


Figure 12.2: Task 12 benchmark – total time for 50,000 searches on 1,000,000 records (log scale; linear is extrapolated).

```
terminal

Reg No: 2400001234 | Name: Your Name | Seed: 1234
Date/Time: 2026-10-12 04:45:40
TASK 12 - 50,000 searches on 1,000,000 records (50% found / 50% not found)
--------------------------------------------------------------------------------
Linear   : measured 200 searches -> 25.372 ms/search ; extrapolated 50,000 searches = 1268.6 s
Binary   : sort 0.4006 s (once) + 50,000 searches 0.2647 s (5.29 us/search) = 0.6653 s
Hash     : build 0.7017 s (once) + 50,000 searches 0.0279 s (0.56 us/search) = 0.7296 s
Workload correctness: 50,000/50,000 answers agree with the oracle
Daily update with 10,000 new records:
  binary: merge into sorted array 0.0476 s | full re-sort 0.3390 s
  hash  : insert new keys 0.0278 s | full rebuild 1.4645 s
Break-even number of searches (preprocessing cost / time saved per search vs linear):
  Binary : 15.79 searches
  Hash   : 27.66 searches
One day (1,200,000 searches): linear ~8.46 h, binary ~6.35 s, hash ~0.67 s of search time
--------------------------------------------------------------------------------
SUMMARY  linear 1268.6 s (extrapolated) | binary 0.6653 s | hash 0.7296 s
```

Figure 12.3: Task 12 benchmark output (50,000 searches on 1,000,000 records, daily update test; reg. no., name, date/time shown).

- **Mathematics.** With $s = 1.2 \times 10^6$ searches per day the per-search term dominates: total $\approx P + s \cdot t$. The hash table's expected $\Theta(1)$ comparisons (≈ 1.4) versus binary search's ≈ 19 comparisons gives a theoretical saving of about 13× in comparisons; measured time saving per search is 9.5×.
- **Preprocessing is paid once a day** (0.70 s rebuild, 0.028 s incremental) and is repaid after ≈ 63,578 searches (≈ 76 minutes of the stated load), i.e. well within the same day.
- **Dataset size.** Cost per search grows with log n for binary search and stays flat for hashing, so the margin widens if the university grows; the table memory (about 1.33 million slots) is small.
- **Honest limits of the evidence.** At today's numbers the absolute gain is small – in my Python implementation both designs need under one second of CPU per hour of load, and for the one-hour benchmark the original design is 0.064 s faster in total because it builds faster. The revised design is justified by per-query cost, scalability and a negligible-cost daily update, not by an indispensable speed-up. If the system must also answer range or ordered queries, or if memory is scarce, the original sorted-array design is a defensible choice and I would retain it (the daily merge of 0.048 s is cheap).
- **Trade-offs of the revised design.** No ordered queries on the hash index (hence the secondary sorted array); extra memory; expected-case guarantee only – a poor hash function or adversarial keys could create long chains (mitigation: prime table size, monitor the longest chain – it was only 7 at $n = 10^6$); searches should use the previous day's index until the new one is complete.

Final decision: modify the original solution – hash-table index rebuilt daily, retaining the sorted array for ordered reports. This follows from the measured costs above: constant expected search time, a one-time daily preprocessing cost that is repaid in under an hour of load, and a tiny cost for the once-a-day update.